

# Computational Complexity - Complete Interview Guide (TOC)

---

A placement-focused deep dive into complexity theory: time/space complexity, P, NP, NP-hard, NP-complete, reductions, SAT/3-SAT, Cook-Levin, PSPACE, the polynomial hierarchy, approximation, pseudo-polynomial and parameterized algorithms, interactive proofs, and probabilistic classes.

---

## 1. Overview

### Definition

**Computational complexity** is the branch of Theory of Computation that studies **how much resource** (time, memory, randomness, communication) a problem *fundamentally* requires to be solved, and **classifies problems** into groups (classes) based on those resource needs.

It is not about "how do I code this," it is about "how hard is this problem, no matter how clever the code is."

Two levels of the same idea:

- **Algorithm analysis** (Big-O): resource cost of *one specific algorithm*.
- **Complexity theory**: the *inherent* difficulty of a *problem* across *all possible algorithms*, and how problems relate to each other.

### Why it matters

- Tells you when to **stop looking for a fast exact algorithm** and switch to approximation, heuristics, or special-case solutions.
- Explains why some everyday problems (scheduling, routing, packing, timetabling) have no known efficient exact solution.
- Underpins **cryptography** - RSA is safe *because* factoring is believed hard.
- Gives a rigorous vocabulary ("this is NP-complete") that instantly communicates difficulty to other engineers.

### Where it is used in real systems

| System           | Where complexity shows up                                     |
|------------------|---------------------------------------------------------------|
| Compilers        | Register allocation = graph coloring (NP-complete)            |
| Databases        | Join order optimization is NP-hard; optimizers use heuristics |
| OS / Cloud       | Task-to-machine scheduling, bin packing (NP-hard)             |
| Cryptography     | Security rests on hardness of factoring / discrete log        |
| Maps / logistics | Route planning = TSP / vehicle routing (NP-hard)              |
| ML               | Training some models, feature selection = NP-hard subproblems |

| System       | Where complexity shows up                                 |
|--------------|-----------------------------------------------------------|
| Verification | SAT/SMT solvers power model checkers and static analyzers |

## Why interviewers ask about it

- It separates candidates who **memorize algorithms** from those who **understand limits**.
- A single "is this NP-complete?" question tests reductions, definitions, and judgment at once.
- Real design work needs it: "This scheduling feature is NP-hard, so I proposed a greedy approximation" is a strong senior answer.
- It is a favorite because the answers reveal depth quickly and are hard to fake.

## 2. Core Idea

### Intuition

Every problem has a **cost floor**. You can write worse algorithms that cost more, but you can never go below the problem's intrinsic difficulty. Complexity theory maps out those floors and, crucially, groups problems so that "solving one efficiently solves a whole family efficiently."

The single most important intuition:

**Verifying** a solution is often much easier than **finding** one.

Checking a completed Sudoku is trivial. Solving a blank one is hard. That gap - find vs. verify - is the entire drama of P vs NP.

### Real-world analogy

Think of a **jigsaw puzzle**:

- **Finding** the arrangement: could take hours (search a huge space).
- **Verifying** a finished puzzle: one glance (does the picture look right?).

If someone hands you a claimed solution (a "certificate"), you check it fast. That is the essence of **NP**: problems whose yes-answers have short, quickly checkable proofs.

### Small example

Problem: **Subset Sum** - given numbers {3, 34, 4, 12, 5, 2} and target 9, is there a subset summing to 9?

- **Finding**: potentially try all  $2^n$  subsets.
- **Verifying**: given the hint {4, 5}, add them → 9. Instant check.

So Subset Sum is in NP: easy to verify a witness, hard (as far as we know) to find one.

### Step-by-step: how to think about any problem's complexity

1. **State it as a decision problem** (yes/no). "Is there a tour shorter than K?" not "find the shortest tour."
2. **Ask: can I verify a claimed yes-answer in polynomial time?** If yes → in NP.

3. **Ask: can I solve it outright in polynomial time?** If yes  $\rightarrow$  in P.
4. **Ask: is every NP problem reducible to it?** If yes  $\rightarrow$  NP-hard.
5. **In NP and NP-hard together**  $\rightarrow$  NP-complete (the hardest problems in NP).
6. If it needs polynomial *memory* but maybe exponential *time*  $\rightarrow$  think PSPACE.

### 3. Important Subtopics

Each subtopic below is treated in depth: what it means, why it matters, an example, and the common interview angle.

#### 3.1 Time and Space Complexity

##### What it means.

- **Time complexity:** number of elementary steps a Turing machine / algorithm takes as a function of input size  $n$ .
- **Space complexity:** amount of memory (tape cells / words) used as a function of  $n$ .
- Measured **asymptotically** with Big-O (upper bound), Big-Omega (lower bound), Big-Theta (tight bound).
- Complexity is defined on the **worst case** unless stated otherwise (also average-case and amortized exist).

**Model matters.** In TOC, the reference model is the **Turing machine**, and "polynomial time" means polynomial in the input length measured in **bits**. This bit-length detail is what makes pseudo-polynomial algorithms (Section 3.13) subtle.

##### Key resource classes (definitions to memorize):

| Class           | Meaning                                                         |
|-----------------|-----------------------------------------------------------------|
| DTIME( $f(n)$ ) | Solvable by a deterministic TM in $O(f(n))$ time                |
| P               | Union of DTIME( $n^k$ ) for all $k \rightarrow$ polynomial time |
| EXPTIME         | DTIME( $2^{\{n^k\}}$ ) $\rightarrow$ exponential time           |
| L (LOGSPACE)    | $O(\log n)$ work space                                          |
| PSPACE          | Polynomial work space                                           |
| NL              | Nondeterministic log space                                      |

**Why it matters.** Everything downstream (P, NP, PSPACE) is defined by bounding one of these two resources. Confusing time with space is a classic error.

**Example.** Binary search: time  $O(\log n)$ , space  $O(1)$  iterative. Merge sort: time  $O(n \log n)$ , space  $O(n)$ . DFS on a graph: time  $O(V+E)$ , space  $O(V)$  for the stack.

##### Interview angle.

- "What is the difference between time and space complexity, and can an algorithm be efficient in one but not the other?" (Yes - e.g. recursion trades stack space for cleaner time.)
  - "Why do we use asymptotic notation instead of exact counts?" (Machine-independence, focus on growth.)
  - "Is space always at most time?" (Yes: you cannot use more cells than steps, so  $\text{SPACE} \subseteq \text{TIME}$  up to the trivial bound; hence  $\text{L} \subseteq \text{P}$ ,  $\text{PSPACE} \subseteq \text{EXPTIME}$ .)
- 

### 3.2 Class P

**What it means.**  $\text{P}$  = the set of **decision problems** solvable by a deterministic Turing machine in **polynomial time**,  $O(n^k)$  for some constant  $k$ .

Informally,  $\text{P}$  = "problems we consider **efficiently solvable / tractable**."

**Why it matters.**

- $\text{P}$  is the practical dividing line between "you can run this at scale" and "you probably cannot."
- **Robust definition:**  $\text{P}$  is the same across all reasonable deterministic models (RAM, TM, multi-tape), thanks to polynomial-time simulation between them - the **Cobham-Edmonds thesis**.
- Closed under composition: a polynomial algorithm calling another polynomial subroutine polynomially many times is still polynomial.

**Example problems in  $\text{P}$ .**

- Sorting, shortest path (Dijkstra/BFS), string matching.
- **Primality testing** (AKS algorithm, 2002) - famously moved *into*  $\text{P}$ .
- Linear programming (Ellipsoid / interior-point methods).
- Matching in graphs, 2-SAT (yes, 2-SAT is in  $\text{P}$  even though 3-SAT is NP-complete).

**Interview angle.**

- "Why is polynomial time the definition of 'efficient' when  $n^{100}$  is slow?" (It is a robust, model-independent, composable boundary; in practice natural polynomial algorithms have small exponents.)
  - "Give an example of a problem that surprised people by being in  $\text{P}$ ." (Primality, linear programming.)
  - Trap: candidates say " $\text{P}$  = fast." Precisely,  $\text{P}$  = polynomial worst-case time on a deterministic machine.
- 

### 3.3 Class NP

**What it means.** Two equivalent definitions - know both.

1. **Verifier definition (most useful):** NP is the set of decision problems for which a **yes**-answer has a **certificate (witness)** that a deterministic machine can **verify in polynomial time**.
2. **Nondeterministic definition (the name's origin):** problems solvable by a **Nondeterministic Turing machine** in **Polynomial time** - the machine "guesses" the certificate and checks it.

NP does **not** mean "non-polynomial." It means **Nondeterministic Polynomial**.

**Why it matters.** NP captures the huge, practically important family of "search for a structure" problems: scheduling, routing, packing, configuration, constraint satisfaction. Thousands of real problems live here.

**Key facts.**

- $P \subseteq NP$  (if you can solve it, you can verify it - just re-solve and compare).
- Whether  $P = NP$  is the biggest open problem in computer science (\$1M Clay prize). Almost everyone believes  $P \neq NP$ .
- The certificate is only required for **yes** instances. NP is *asymmetric* (this is why Co-NP exists - Section 3.9).

**Example.** Hamiltonian Cycle: "Does this graph have a cycle visiting every vertex exactly once?" Certificate = the ordering of vertices. Verify in  $O(n)$ : check consecutive edges exist and all vertices used once.

**Interview angle.**

- "Define NP." Give the **verifier** definition first - it is clearer and impresses.
- "Is NP the set of unsolvable/hard problems?" No - NP contains all of P too.
- "What does the certificate look like for problem X?" Be ready to name it (a satisfying assignment, a tour, a subset, a coloring).

**3.4 NP-hard and NP-complete****What they mean.**

- **NP-hard:** A problem **H** is NP-hard if **every** problem in NP reduces to it in polynomial time. Intuition: "at least as hard as everything in NP." H need **not** be in NP and need **not** even be a decision problem (optimization versions are NP-hard).
- **NP-complete:** **C** is NP-complete if (1)  $C \in NP$  and (2) **C** is NP-hard. These are the **hardest problems inside NP**.

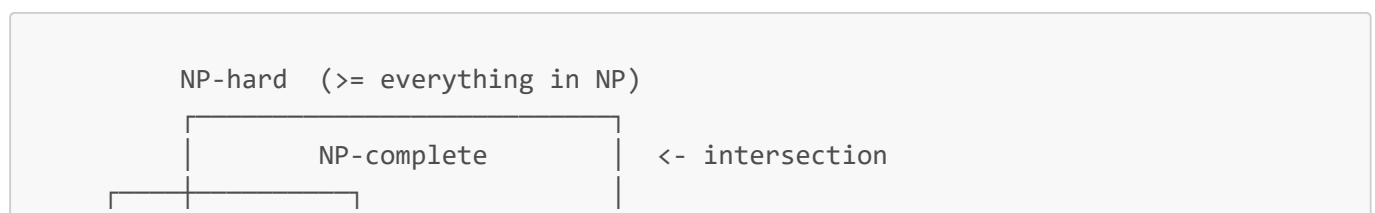
So NP-complete = "in NP"  $\cap$  "NP-hard".

**Why it matters.**

- If **any** NP-complete problem has a polynomial algorithm, then  $P = NP$  and *all* of NP collapses to P. That is why NP-completeness is a "master switch."
- It gives you license to stop: "This is NP-complete, so no known polynomial exact algorithm exists; here is my approximation/heuristic."

**Example.**

- **NP-complete:** SAT, 3-SAT, Clique, Vertex Cover, Hamiltonian Cycle, Subset Sum, Graph Coloring, the *decision* TSP ("is there a tour  $\leq K$ ?").
- **NP-hard but not (known to be) in NP:** the *optimization* TSP ("find the shortest tour"), Halting Problem (NP-hard and undecidable, definitely not in NP).

**Mental model (the map):**

|    |        |               |                       |
|----|--------|---------------|-----------------------|
| NP | 3-SAT  | Halting Prob  | <- NP-hard, not in NP |
|    | Clique | (undecidable) |                       |
| P  | ...    |               |                       |

### Interview angle.

- "Difference between NP-hard and NP-complete?" NP-complete must *also* be in NP (verifiable); NP-hard need not be.
- "Is the Halting Problem NP-complete?" No - it is NP-hard but not in NP (it is undecidable, so no verifier).
- "How do you prove a new problem NP-complete?" (1) show it is in NP; (2) reduce a known NP-complete problem *to* it. Direction matters (next section).

## 3.5 Polynomial-time Reductions

**What it means.** A **reduction** transforms instances of problem **A** into instances of problem **B** so that the yes/no answer is preserved, using a **polynomial-time** transformation. Written  $A \leq_p B$  ("A reduces to B").

Reading  $A \leq_p B$ : **B is at least as hard as A**. If you can solve B efficiently, you can solve A efficiently (transform, then solve B).

The standard type is the **many-one (Karp) reduction**: map instance of A  $\rightarrow$  one instance of B, preserving the answer. (A more general kind is the **Turing/Cook reduction**, which may call B multiple times.)

### Why it matters.

- Reductions are the **engine** of complexity theory. NP-completeness proofs are almost entirely reductions.
- They **transfer hardness**: if A is NP-hard and  $A \leq_p B$ , then B is NP-hard.
- They also **transfer algorithms**: a fast solver for B gives a fast solver for A.

**Direction is everything (the #1 mistake).** To prove **B is NP-hard**, reduce a **known-hard** problem *into* B:  $\text{known-NPC} \leq_p B$ . Reducing B into an easy problem proves nothing about B's hardness.

### Example. Reducing 3-SAT $\leq_p$ Independent Set:

- For each clause  $(a \vee b \vee c)$  build a triangle of 3 vertices (one per literal).
- Add an edge between any two vertices that are complementary literals ( $x$  and  $\neg x$ ).
- Ask: is there an independent set of size = number of clauses  $m$ ?
- A size- $m$  independent set must pick exactly one literal per clause (triangle) with no conflicts  $\rightarrow$  a satisfying assignment. The construction is clearly polynomial.

### Interview angle.

- "Which way does the reduction go to prove hardness?" Known-hard problem  $\rightarrow$  your problem.
- "What must a valid reduction preserve?" The answer (yes maps to yes, no maps to no) and run in polynomial time.

- "Difference between Karp and Cook reductions?" Karp = single transformation (many-one); Cook = polynomial-time algorithm that may query B as an oracle multiple times.
- 

### 3.6 SAT and 3-SAT

#### What they mean.

- **SAT (Boolean Satisfiability):** Given a Boolean formula (variables, AND, OR, NOT), is there an assignment of true/false making it **true**?
- **3-SAT:** SAT restricted to **CNF** (conjunctive normal form = AND of OR-clauses) where **each clause has exactly 3 literals**. Example:  $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$ .

#### Why they matter.

- **SAT was the first problem proven NP-complete** (Cook-Levin, 1971). It is the "root" from which most other NP-completeness proofs grow by reduction.
- **3-SAT** is the workhorse: it is still NP-complete but has a rigid, uniform structure that makes reductions to other problems clean. Most NP-completeness proofs start from 3-SAT.
- SAT solvers (DPLL, CDCL) are real, fast-in-practice tools powering hardware verification, planning, and program analysis - despite worst-case exponential behavior.

#### The 2 vs 3 cliff (a favorite question):

- **2-SAT** (2 literals per clause) is in **P** - solvable via implication graphs and strongly connected components.
- **3-SAT** is **NP-complete**.
- This sharp jump from 2 to 3 is a classic "why?" question. Reason: 2-SAT clauses are equivalent to implications ( $\neg a \rightarrow b$ ), giving a graph structure solvable in linear time; 3-clauses lose that pairwise implication structure.

#### Example. $(x \vee y) \wedge (\neg x \vee y) \wedge (\neg y)$ :

- Third clause forces  $y = \text{false}$ .
- First clause then needs  $x = \text{true}$ ; second clause  $(\neg x \vee y) = (\text{false} \vee \text{false}) = \text{false}$ . Contradiction  $\rightarrow$  **unsatisfiable**.

#### Interview angle.

- "Why is 3-SAT so important?" First NP-complete via Cook-Levin; canonical starting point for reductions.
  - "Is 2-SAT NP-complete?" No, it is in P (implication graph + SCC).
  - "Can any SAT be turned into 3-SAT?" Yes - any CNF converts to 3-CNF in polynomial time (split long clauses with new variables, pad short ones), so 3-SAT is as hard as general CNF-SAT.
- 

### 3.7 Cook-Levin Theorem (the idea)

**What it says. SAT is NP-complete.** (Proven independently by Stephen Cook (1971) and Leonid Levin.) It was the **first** problem shown NP-complete and it bootstraps the entire theory.

**Why it matters.** Once you have *one* NP-complete problem, you get thousands more just by reduction (**SAT  $\leq_p$  everything else**). Cook-Levin is that first domino.

**The core idea (know this, not the full proof):**

- Take **any** problem **A** in NP. By definition, a nondeterministic TM **M** verifies it in polynomial time, say  $p(n)$  steps.
- The entire computation of **M** on input **x** can be laid out as a **tableau**: a grid where row **i** = the machine's configuration (tape contents, head position, state) at step **i**. The grid is  $p(n) \times p(n)$  - polynomial size.
- Introduce **Boolean variables** encoding "cell **(i,j)** holds symbol **s**", "head is at position **j** at time **i**", "state is **q** at time **i**".
- Write **clauses** enforcing:
  - The first row = the correct starting configuration for **x**.
  - Each row **legally follows** the previous one under M's transition rules (local consistency between adjacent cells).
  - Some row is an **accepting** configuration.
- This formula is polynomial-size and is **satisfiable if and only if** M has an accepting computation on **x**, i.e. **x** is a yes-instance.
- So **A  $\leq_p$  SAT**. Since **A** was arbitrary in NP, **every** NP problem reduces to SAT  $\rightarrow$  SAT is NP-hard. SAT is obviously in NP (a satisfying assignment is the certificate). Therefore SAT is **NP-complete**. ■ (idea)

**One-line summary.** "A nondeterministic polynomial computation can be *encoded* as a Boolean formula whose satisfiability equals acceptance; so any NP problem becomes a SAT instance."

**Interview angle.**

- "What is the significance of Cook-Levin?" First NP-complete problem; foundation for all other NP-completeness proofs.
- "Sketch the idea." Encode the NTM's computation tableau as a Boolean formula (start, transition, accept clauses).
- Do **not** try to recite the full formal proof - state the tableau/encoding idea clearly and confidently.

### 3.8 Common NP-complete Problems

**Why memorize a catalog.** Interviewers ask "is X NP-complete?" and expect instant recognition. Knowing the canonical list lets you (a) answer directly and (b) pick a good problem to reduce *from*.

| Problem                | Decision question                                   | Certificate           |
|------------------------|-----------------------------------------------------|-----------------------|
| SAT / 3-SAT            | Is the Boolean formula satisfiable?                 | Satisfying assignment |
| Clique                 | Is there a complete subgraph of size k?             | The k vertices        |
| Independent Set        | Is there a set of k mutually non-adjacent vertices? | The k vertices        |
| Vertex Cover           | Is there a set of k vertices covering all edges?    | The k vertices        |
| Hamiltonian Cycle/Path | Is there a cycle/path visiting every vertex once?   | The ordering          |
| TSP (decision)         | Is there a tour of length $\leq K$ ?                | The tour              |

| Problem                | Decision question                             | Certificate      |
|------------------------|-----------------------------------------------|------------------|
| Subset Sum             | Is there a subset summing to target $T$ ?     | The subset       |
| Knapsack (decision)    | Value $\geq V$ within weight $\leq W$ ?       | The chosen items |
| Graph Coloring         | Can the graph be colored with $k$ colors?     | The coloring     |
| Set Cover              | Cover the universe with $\leq k$ sets?        | The chosen sets  |
| Partition              | Split the multiset into two equal-sum halves? | One half         |
| Bin Packing (decision) | Fit items into $\leq k$ bins?                 | The assignment   |

### Useful reduction relationships (the "family tree"):



(Clique, Independent Set, Vertex Cover are the "same" problem under complementation - a quick reduction between them.)

### Interview angle.

- "Name some NP-complete problems." Rattle off 5-6 from different families (logic, graph, number).
- "Clique vs Independent Set vs Vertex Cover relationship?" A set  $S$  is a clique in  $G$  iff  $S$  is an independent set in the complement graph; and  $S$  is a vertex cover iff its complement is an independent set. All polynomial-time reducible to each other.
- "Is Knapsack NP-complete but also solvable by DP?" Yes - see pseudo-polynomial (Section 3.13). This is a classic trap.

## 3.9 Co-NP

**What it means.** **Co-NP** is the set of problems whose **no**-answers have short, polynomial-time-verifiable certificates. Equivalently,  $L \in \text{Co-NP}$  iff its complement  $\bar{L} \in \text{NP}$ .

- NP: easy to certify **yes**.
- Co-NP: easy to certify **no**.

**Why it matters.** Highlights NP's asymmetry. Some problems are naturally "prove it is impossible," which is a Co-NP flavor.

**Example.**

- **TAUTOLOGY** ("is this formula true for every assignment?") is Co-NP-complete. A *no* is easy to certify (one falsifying assignment); a *yes* seems to need checking all assignments.
- **UNSAT** ("is this formula unsatisfiable?") is the complement of SAT  $\rightarrow$  Co-NP-complete.

**Key facts.**

- $P \subseteq NP \cap Co-NP$  (deterministic solvers give certificates both ways).
- Open whether  $NP = Co-NP$  (believed **no**). If any NP-complete problem is in Co-NP, then  $NP = Co-NP$ .
- **Primality** is in  $NP \cap Co-NP$  (and later shown to be in P) - a historically important example.

**Interview angle.**

- "What is Co-NP?" Complements of NP problems / easy-to-verify *no* answers.
- "Give a Co-NP-complete problem." TAUTOLOGY or UNSAT.
- "Is  $P \subseteq NP \cap Co-NP$ ?" Yes.
- Trap: "Co-NP means not in NP." Wrong - P sits inside both; they overlap heavily.

**3.10 PSPACE**

**What it means.** **PSPACE** = problems solvable using a **polynomial amount of memory**, with **no restriction on time** (can take exponential time as long as space stays polynomial).

**Why it matters.** Space is reusable; time is not. So bounding space is a much weaker restriction, making PSPACE a large, powerful class. Many **two-player games** and **quantified logic** problems live here.

**Known relationships (memorize this chain):**

$$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$$

(and  $Co-NP \subseteq PSPACE$ )

- $NP \subseteq PSPACE$ : you can enumerate all polynomial-size certificates one at a time, reusing space.
- $PSPACE \subseteq EXPTIME$ : a machine with polynomial space  $s$  has at most  $2^{O(s)}$  distinct configurations, so it must halt within exponentially many steps.
- **Savitch's theorem:**  $NPSPACE = PSPACE$  (nondeterminism does not add power for space) - deterministic space  $O(s^2)$  simulates nondeterministic space  $O(s)$ . This is why we do not talk much about "NPSPACE."

**Canonical PSPACE-complete problem.** **QBF / TQBF** (Quantified Boolean Formula): is  $\exists x_1 \forall x_2 \exists x_3 \dots \phi$  true? The alternating quantifiers model perfect play in games ("there exists my move such that for all opponent moves, there exists my move ..."). Generalized versions of Chess, Go, and Reversi are PSPACE-hard (or harder).

**Example.** Deciding whether the first player has a **winning strategy** in a generalized board game is typically PSPACE-complete.

**Interview angle.**

- "How does PSPACE relate to P and NP?"  $P \subseteq NP \subseteq PSPACE$ ; equality with NP is open.
  - "Give a PSPACE-complete problem." TQBF / two-player games.
  - "Why is NP inside PSPACE?" Reuse space to try each certificate sequentially.
  - "What is Savitch's theorem?"  $NPSpace = PSPACE$  (space is robust to nondeterminism).
- 

### 3.11 Polynomial Hierarchy (basics)

**What it means.** The **Polynomial Hierarchy (PH)** generalizes NP and Co-NP into an infinite tower of classes defined by **alternating quantifiers** ( $\exists, \forall$ ) with a polynomial-time predicate.

- Level 0:  $\Sigma_0 = \Pi_0 = P$ .
- $\Sigma_1 = NP$  (one  $\exists$  block):  $\exists$  certificate, poly-check.
- $\Pi_1 = Co-NP$  (one  $\forall$  block):  $\forall$  inputs, poly-check.
- $\Sigma_2: \exists \dots \forall \dots$  poly-check (two alternations).
- $\Pi_2: \forall \dots \exists \dots$  poly-check.
- ... and so on.  $PH = \cup \Sigma_k$ .

**Why it matters.**

- Models problems more complex than NP but still within PSPACE, e.g. "is there a strategy that beats **all** opponents?" ( $\exists$  then  $\forall$ ).
- A theoretical tool: many results are stated as "unless the polynomial hierarchy collapses," which is considered very unlikely.

**Key facts.**

- $PH \subseteq PSPACE$ .
- **Collapse:** if  $\Sigma_k = \Sigma_{k+1}$  for some  $k$ , the whole hierarchy collapses to that level. If  $P = NP$ , PH collapses to P.
- Belief: PH is a strict infinite hierarchy (does not collapse).

**Example.** "Is this Boolean circuit the **smallest** one computing its function?" - "there exists no smaller circuit that agrees on all inputs" - a  $\forall/\exists$  alternation problem sitting in the second level of PH.

**Interview angle.**

- "What is the polynomial hierarchy?" A tower generalizing NP ( $\exists$ ) and Co-NP ( $\forall$ ) by alternating quantifiers.
  - "What is at level 1?"  $\Sigma_1 = NP, \Pi_1 = Co-NP$ .
  - "What happens if  $P = NP$ ?" PH collapses to P.
  - This is usually a *bonus/senior* topic; a crisp 2-sentence answer is plenty.
- 

### 3.12 Approximation Algorithms (basics)

**What it means.** For NP-hard **optimization** problems, we give up on exact optimum and design **polynomial-time algorithms that provably come close**. An algorithm is an  **$\alpha$ -approximation** if its solution is always within factor  $\alpha$  of optimal ( $\alpha \geq 1$  for minimization:  $ALG \leq \alpha \cdot OPT$ ).

**Why it matters.** This is the *practical response* to NP-hardness. You cannot solve TSP optimally at scale, but a 1.5-approximation (Christofides) is often good enough for real routing.

### Key concepts.

- **Approximation ratio:** the guaranteed factor from optimal.
- **PTAS** (Polynomial-Time Approximation Scheme): for any  $\epsilon > 0$ , a  $(1+\epsilon)$ -approximation in time polynomial in  $n$  (but possibly exponential in  $1/\epsilon$ ).
- **FPTAS** (Fully PTAS): time polynomial in *both*  $n$  and  $1/\epsilon$  - the gold standard. Knapsack has an FPTAS.
- **Inapproximability:** some problems are hard even to approximate. E.g. general TSP has no constant-factor approximation unless  $P=NP$ ; **MAX-3SAT** cannot be approximated better than  $7/8$  unless  $P=NP$  (PCP theorem).

### Examples.

- **Vertex Cover:** simple 2-approximation (repeatedly pick both endpoints of an uncovered edge).
- **Metric TSP:** Christofides gives 1.5-approximation.
- **Set Cover:** greedy gives an  $O(\log n)$ -approximation (and that is essentially optimal).
- **Knapsack:** FPTAS via scaling values.

### Interview angle.

- "What do you do when a problem is NP-hard?" Approximation / heuristics / restrict to special cases / exponential exact for small  $n$ .
- "What is a 2-approximation?" Result never worse than  $2\times$  optimal.
- "Difference between PTAS and FPTAS?" FPTAS is polynomial in  $1/\epsilon$  too; PTAS may be exponential in  $1/\epsilon$ .
- "Can every NP-hard problem be approximated well?" No - inapproximability results (PCP theorem) show some cannot, unless  $P=NP$ .

## 3.13 Pseudo-polynomial Algorithms

**What it means.** An algorithm is **pseudo-polynomial** if its running time is polynomial in the **numeric value** of the input, but **exponential in the input's bit-length** (the number of digits/bits used to write those numbers).

The classic case: **0/1 Knapsack / Subset Sum** dynamic programming runs in  $O(n \cdot W)$  where  $W$  is the capacity (a *value*). But  $W$  is written using only  $\log W$  bits, so  $O(n \cdot W) = O(n \cdot 2^{\{\log W\}})$  - **exponential in the input size measured in bits**.

### Why it matters.

- Resolves the apparent paradox: "Knapsack is NP-complete, yet I have a DP that solves it - is  $P=NP$ ?!" No. The DP is pseudo-polynomial, not truly polynomial. It is fast only when numbers are small.
- Distinguishes **weakly NP-complete** problems (Knapsack, Subset Sum, Partition - solvable in pseudo-poly time) from **strongly NP-complete** problems (TSP, 3-SAT, Graph Coloring - NP-hard even when all numbers are small/polynomially bounded, so no pseudo-poly algorithm unless  $P=NP$ ).

**Example.** Subset Sum with target  $T$ :

- DP table over reachable sums:  $O(n \cdot T)$  time.

- If  $T = 10$  and  $n = 100$ : fast. If  $T = 2^{64}$ : hopeless. Same  $n$ , wildly different runtime - because the runtime tracks the *value*, not the count.

### Interview angle.

- "Knapsack has an  $O(nW)$  DP - doesn't that make it polynomial / prove  $P=NP$ ?" No.  $W$  is exponential in its bit-length; the algorithm is pseudo-polynomial.
- "What is the difference between weakly and strongly NP-complete?" Weakly = has a pseudo-poly algorithm (numbers matter); strongly = hard even with small numbers.
- "When is a pseudo-poly algorithm actually good?" When the numeric values are small (bounded by a polynomial in  $n$ ).

## 3.14 Parameterized Complexity

**What it means.** A finer-grained lens: instead of measuring difficulty by input size  $n$  alone, add a **parameter**  $k$  (some structural quantity - solution size, treewidth, etc.) and ask whether the exponential blowup can be **confined to**  $k$  rather than  $n$ .

- **FPT (Fixed-Parameter Tractable)**: solvable in  $f(k) \cdot n^{O(1)}$  time - the exponential part depends *only* on the parameter  $k$ , and  $n$  appears only polynomially. Efficient when  $k$  is small.
- Contrast with  $n^{O(k)}$  (like brute force), which is **not** FPT (the  $n$  exponent grows with  $k$ ).

**Why it matters.** Many NP-hard problems are FPT for natural parameters - so they are genuinely tractable in practice when the parameter is small, even though they are NP-hard in general. This is a nuanced, senior-level insight that reframes "NP-hard = hopeless."

### Examples.

- **Vertex Cover** parameterized by cover size  $k$ : FPT, solvable in  $O(2^k \cdot n)$  via bounded search tree (branch on each edge: include one endpoint or the other). Practical for small  $k$ .
- **k-Clique**: believed **not** FPT - it is  $W[1]$ -hard (the parameterized analogue of NP-hard). Best known is  $n^{O(k)}$ .
- Problems on graphs of bounded **treewidth** are often FPT.

**The W-hierarchy.**  $FPT \subseteq W[1] \subseteq W[2] \subseteq \dots$  - the parameterized analogue of P vs NP.  $W[1]$ -hardness is evidence a problem is *not* FPT.

### Interview angle.

- "What is fixed-parameter tractability?" Running time  $f(k) \cdot \text{poly}(n)$  - exponential blowup limited to parameter  $k$ .
- "Give an FPT problem." Vertex Cover parameterized by  $k$ :  $O(2^k \cdot n)$ .
- "Difference between  $f(k) \cdot n^c$  and  $n^{f(k)}$ ?" Only the first is FPT; the second (like k-Clique brute force) is not.
- Usually a *bonus* topic for strong candidates or research-flavored roles.

## 3.15 Interactive Proofs

**What it means.** A generalization of NP where a computationally powerful but untrusted **Prover** tries to convince a randomized, polynomial-time **Verifier** that a statement is true, through a **conversation (multiple rounds)** using **randomness**. The verifier accepts true statements (completeness) and rejects false ones with high probability (soundness).

- NP = a *single* message (the certificate) from an all-powerful prover to a *deterministic* verifier.
- IP = many rounds + randomness. This is strictly more powerful.

#### Why it matters.

- Landmark result: **IP = PSPACE** (Shamir, 1992). Interaction + randomness lets a poly-time verifier check membership in *any* PSPACE problem.
- **Zero-knowledge proofs** (a special kind of interactive proof) are the theoretical basis of modern cryptographic protocols, authentication, and **blockchain scaling (zk-SNARKs / zk-rollups)**.
- **PCP theorem** ( $\text{NP} = \text{PCP}(\log n, O(1))$ ): every NP proof can be rewritten so the verifier checks only a **constant number of randomly chosen bits** - the foundation of hardness-of-approximation results.

**Example (intuition) - Graph Non-Isomorphism.** Two graphs G1, G2. Verifier randomly picks one, scrambles (randomly relabels) it, and asks the all-powerful prover "which original was this?" If the graphs are truly non-isomorphic, the prover can always tell and answers correctly every round. If they were actually isomorphic, the prover cannot distinguish and is caught with probability 1/2 each round → soundness amplifies with repetition.

#### Interview angle.

- "What is an interactive proof / how does it differ from NP?" Multiple rounds + randomness + interaction vs single static certificate.
- "What is the famous IP result?" **IP = PSPACE**.
- "What is a zero-knowledge proof?" An interactive proof that convinces the verifier a statement is true *without revealing why* (no info beyond validity leaks). Basis of zk-SNARKs.
- Advanced/bonus topic - a short accurate answer stands out.

### 3.16 Probabilistic Complexity Classes

**What it means.** Classes of problems solvable by algorithms that use **randomness** (coin flips) and are allowed a small, controllable **probability of error**.

| Class        | Meaning                                | Error type                                                                                           |
|--------------|----------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>BPP</b>   | Bounded-error Probabilistic Polynomial | Two-sided error $\leq 1/3$ on both yes and no; amplifiable to negligible by repetition               |
| <b>RP</b>    | Randomized Polynomial                  | One-sided: never says yes wrongly; may miss a yes with prob $\leq 1/2$                               |
| <b>Co-RP</b> | Complement of RP                       | One-sided the other way (never wrongly says no)                                                      |
| <b>ZPP</b>   | Zero-error Probabilistic Poly          | Always correct, <i>expected</i> polynomial time (Las Vegas). <b>ZPP = RP <math>\cap</math> Co-RP</b> |

| Class | Meaning                  | Error type                                                                 |
|-------|--------------------------|----------------------------------------------------------------------------|
| PP    | Probabilistic Polynomial | Error $< 1/2$ but <i>not</i> bounded away from it (not practically usable) |

### Monte Carlo vs Las Vegas:

- **Monte Carlo** (BPP/RP): fixed runtime, small chance of wrong answer.
- **Las Vegas** (ZPP): always correct, runtime is random (fast in expectation). Example: randomized Quicksort.

### Why it matters.

- Randomized algorithms are often **simpler and faster** than deterministic ones (Miller-Rabin primality, randomized min-cut, hashing, Monte Carlo integration).
- Big question: **is  $P = BPP$ ?** Widely believed **yes** (randomness gives at most polynomial speedup; strong evidence from derandomization / pseudorandom generators). This is why AKS (deterministic primality in P) was expected but still celebrated.

### Relationships.

$$P \subseteq ZPP \subseteq RP \subseteq BPP \subseteq PP \subseteq PSPACE$$

$$RP \subseteq NP \qquad BPP \subseteq \Sigma_2 \cap \Pi_2 \text{ (in PH)}$$

**Example. Miller-Rabin primality test:** pick random witnesses; a composite is exposed with high probability each round. Runs in polynomial time with tunable error - a Co-RP / BPP-style algorithm used in real crypto libraries (before/alongside AKS, because it is much faster).

### Interview angle.

- "What is BPP?" Polynomial-time with two-sided bounded error, amplifiable to near-zero.
- "Monte Carlo vs Las Vegas?" Fixed time / maybe wrong vs always right / random time.
- "Is  $P = BPP$ ?" Open, but widely believed yes.
- "Name a real randomized algorithm." Miller-Rabin, randomized Quicksort, Karger's min-cut.

## 4. Real-World Example

### Scenario: A cloud scheduler placing containers on servers (bin packing).

You are building a Kubernetes-style scheduler. Each server has a fixed CPU/RAM capacity; each container needs some CPU/RAM. Goal: **pack all containers onto the fewest servers** (to save cost).

1. **This is Bin Packing** - a known **NP-hard / NP-complete (decision version)** problem. There is no known polynomial algorithm for the exact optimum.
2. **You do not attempt exact optimization** at scale (thousands of containers). Recognizing NP-hardness saves you from chasing an impossible perfect algorithm.
3. **You use an approximation/heuristic: First-Fit-Decreasing** (sort items descending, place each in the first server that fits). It is a fast, provably good approximation (within  $\sim 11/9$  of optimal for bin packing).

4. **Numbers are small** (CPU in millicores), so even a **pseudo-polynomial DP** could be viable for a single node's subset selection.
5. **If the parameter is small** (e.g. only  $k$  distinct container sizes), **parameterized** techniques could give exact answers efficiently.

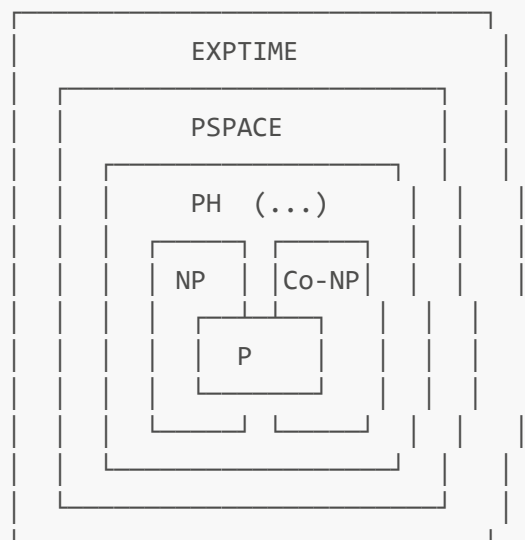
The whole engineering decision - "heuristic, not exact; here is the guarantee" - flows directly from knowing the complexity class. That is complexity theory paying rent in production.

Other quick mappings:

- **Compiler register allocation** → graph coloring (NP-complete) → heuristic coloring + spilling.
- **Database query optimizer** → join ordering (NP-hard) → dynamic programming for small joins, greedy/genetic for large.
- **Cryptography (TLS/RSA)** → security *relies on* integer factoring having no known efficient algorithm.
- **SAT/SMT solvers** in static analyzers and CI verification → NP-complete core, but engineered to be fast on real instances.

## 5. Diagrams / Mental Models

The class containment map (the "one diagram to rule them all")



Known:  $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ ,  $P \subseteq Co-NP \subseteq PSPACE$   
 Known strict:  $P \subsetneq EXPTIME$  (time hierarchy theorem)  
 Open:  $P$  vs  $NP$ ,  $NP$  vs  $Co-NP$ ,  $NP$  vs  $PSPACE$ ,  $P$  vs  $BPP$

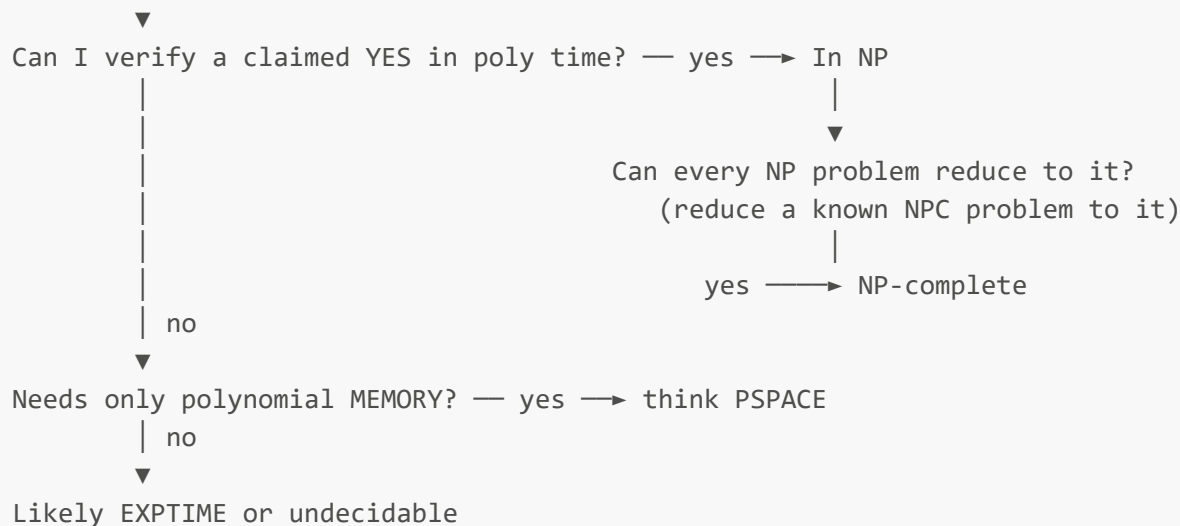
Decision flowchart: "how hard is my problem?"

Start: state it as a yes/no decision problem



Can I solve it in polynomial time? — yes → In  $P$  (done, it's tractable)

↓ no / unknown



## How to prove NP-completeness (recipe)

To show problem X is NP-complete:

Step 1: Show  $X \in \text{NP}$

→ describe a certificate + poly-time verifier.

Step 2: Show X is NP-hard

→ pick a known NP-complete problem Y (usually 3-SAT)

→ build poly-time reduction  $Y \leq_p X$

→ argue: Y is YES  $\Leftrightarrow$  X is YES

Done. (Direction: KNOWN-hard → YOUR problem)

## Resource intuition table

| Resource restricted                   | Class             |
|---------------------------------------|-------------------|
| Poly time, deterministic              | P                 |
| Poly time, verify a witness           | NP                |
| Poly time, verify a "no"              | Co-NP             |
| Poly <i>space</i> , any time          | PSPACE            |
| Poly time + randomness, bounded error | BPP               |
| Poly value (not bit-length)           | Pseudo-polynomial |
| $f(k) \cdot \text{poly}(n)$           | FPT               |

## 6. Common Interview Questions

### Q1. What does NP stand for, and what is a common misconception?

- **Answer:** Nondeterministic Polynomial time. The misconception is that it means "non-polynomial" or "not solvable in polynomial time." NP is the class of problems whose yes-answers are verifiable in

polynomial time; it *contains* all of P.

- **Interviewer expects:** the verifier definition, and awareness that  $P \subseteq NP$ .
- **Common mistake:** "NP = hard/unsolvable problems."

## Q2. What is the difference between NP-hard and NP-complete?

- **Answer:** NP-complete = in NP **and** NP-hard. NP-hard = at least as hard as every NP problem, but need not be in NP (may not even be a decision problem or may be undecidable). NP-complete are the hardest problems *inside* NP.
- **Expects:** the "in NP" extra condition; an example of NP-hard-but-not-in-NP (Halting Problem, optimization TSP).
- **Mistake:** treating the two as synonyms.

## Q3. If one NP-complete problem is solved in polynomial time, what happens?

- **Answer:** Then  $P = NP$  - because every NP problem reduces to it in polynomial time, so all of NP becomes polynomial.
- **Expects:** understanding of reductions and the collapse.
- **Mistake:** saying "only that problem becomes easy."

## Q4. Which way must a reduction go to prove problem X is NP-hard?

- **Answer:** Reduce a **known NP-complete** problem **to X** (**known**  $\leq_p$  X). This shows X is at least as hard as the known one.
- **Expects:** correct direction and the phrase "at least as hard."
- **Mistake:** reducing X to an easy problem (proves nothing about X's hardness).

## Q5. Why is 2-SAT in P but 3-SAT NP-complete?

- **Answer:** 2-SAT clauses ( $a \vee b$ ) are equivalent to implications ( $\neg a \rightarrow b$ ), giving an implication graph solvable in linear time via strongly connected components. 3-clauses lack this pairwise-implication structure, and 3-SAT is the canonical NP-complete problem.
- **Expects:** mention of implication graph / SCC for 2-SAT.
- **Mistake:** thinking all SAT variants are equally hard.

## Q6. What was the first problem proven NP-complete and by whom?

- **Answer:** **SAT (Boolean satisfiability)**, by the **Cook-Levin theorem** (Cook 1971, Levin independently). It bootstrapped all other NP-completeness proofs via reduction.
- **Expects:** the tableau/encoding idea if pushed.
- **Mistake:** naming 3-SAT or TSP.

## Q7. Knapsack has an $O(nW)$ DP - doesn't that prove $P = NP$ ?

- **Answer:** No. That DP is **pseudo-polynomial**:  $W$  is exponential in its bit-length ( $\log W$  bits). It is polynomial in the *value*  $W$ , not the input *size*. Knapsack is only **weakly** NP-complete.
- **Expects:** bit-length vs value distinction; term "pseudo-polynomial."
- **Mistake:** believing  $O(nW)$  is genuinely polynomial.

## Q8. What is Co-NP? Give an example.

- **Answer:** Problems whose *no*-answers have poly-verifiable certificates (complements of NP problems). Example: **TAUTOLOGY** or **UNSAT** (Co-NP-complete).  $P \subseteq NP \cap \text{Co-NP}$ .
- **Expects:** the "certify no" idea and a concrete Co-NP-complete example.
- **Mistake:** "Co-NP = not in NP."

#### Q9. How does PSPACE relate to P and NP? Give a PSPACE-complete problem.

- **Answer:**  $P \subseteq NP \subseteq PSPACE$ . PSPACE = poly memory, unbounded time. Canonical PSPACE-complete: **TQBF** (quantified Boolean formula); generalized two-player games are PSPACE-hard.
- **Expects:** the containment chain and TQBF/games.
- **Mistake:** confusing space and time restrictions.

#### Q10. What do you do in practice when a problem is NP-hard?

- **Answer:** Options: (1) **approximation** algorithms with provable ratios; (2) **heuristics / metaheuristics** (greedy, genetic, simulated annealing); (3) **exact exponential** algorithms for small  $n$ ; (4) exploit **special structure** (bounded parameter  $\rightarrow$  FPT, small numbers  $\rightarrow$  pseudo-poly DP); (5) use industrial **SAT/ILP solvers** that are fast on real instances.
- **Expects:** at least approximation + heuristics + special cases.
- **Mistake:** "give up" or "it's unsolvable."

#### Q11. Is $P = NP$ , and what is your intuition?

- **Answer:** It is **open** (Clay Millennium problem). The consensus belief is  $P \neq NP$  - because finding solutions to thousands of natural problems seems fundamentally harder than verifying them, and decades of effort found no polynomial algorithm for any NP-complete problem.
- **Expects:** "open problem," correct belief, the find-vs-verify intuition.
- **Mistake:** claiming it is proven either way.

#### Q12. What is a certificate/witness? Give one for Hamiltonian Cycle.

- **Answer:** A short piece of data that lets a verifier confirm a yes-instance in polynomial time. For Hamiltonian Cycle: the **ordering of vertices**; verify each consecutive pair is an edge and every vertex appears once.
- **Expects:** definition + concrete certificate.
- **Mistake:** describing an algorithm to *find* it rather than to *verify* it.

## 7. Deep-Dive Questions

#### D1. Sketch how the Cook-Levin theorem encodes an NP computation as a Boolean formula.

- Any NP problem has a nondeterministic TM verifying it in  $p(n)$  steps. Represent the computation as a  $p(n) \times p(n)$  **tableau** (rows = configurations over time). Introduce Boolean variables for "cell  $(i,j)$  holds symbol  $s$ ," head position, and state. Add polynomial-size **clauses** enforcing: correct start row, legal local transitions between adjacent rows (a constant-size window constraint), and an accepting final row. The formula is satisfiable **iff** the machine accepts. Hence every NP problem  $\leq_p$  SAT, so SAT is NP-hard; SAT  $\in$  NP too  $\rightarrow$  NP-complete.

#### D2. Why does "polynomial space" collapse nondeterminism (Savitch), but "polynomial time" (probably) does not?

- **Savitch's theorem** shows  $\text{NSPACE}(s) \subseteq \text{DSpace}(s^2)$  via a recursive reachability procedure ( $\text{REACH}(a, b, t)$ : is there a path from config  $a$  to  $b$  within  $t$  steps?) that reuses space by splitting on a midpoint. Space is **reusable**, so the quadratic blowup is affordable. Time is **not reusable** - a deterministic machine simulating nondeterministic *time* seems to require exploring exponentially many branches, and no polynomial simulation is known (that is exactly the  $P$  vs  $NP$  question). Result:  $\text{NPSPACE} = \text{PSPACE}$ , but  $P \stackrel{?}{=} NP$  stays open.

### D3. What is the significance of the PCP theorem, and how does it connect to approximation?

- The **PCP theorem** states  $NP = \text{PCP}(\log n, O(1))$ : every NP proof can be encoded so a verifier reads only a **constant number of randomly chosen bits** and still catches false proofs with high probability. Its power is in **hardness of approximation**: it implies that for problems like MAX-3SAT, achieving an approximation ratio better than a specific threshold (7/8 for MAX-3SAT) is itself NP-hard. So PCP draws the line between what can and cannot be approximated well, assuming  $P \neq NP$ .

### D4. Explain the polynomial hierarchy and what "collapse" means.

- PH stacks classes by alternating quantifiers over a poly-time predicate:  $\Sigma_1 = NP (\exists)$ ,  $\Pi_1 = \text{Co-NP} (\forall)$ ,  $\Sigma_2 = \exists\forall$ ,  $\Pi_2 = \forall\exists$ , etc. Each added quantifier alternation is believed to add real power. A **collapse** happens if two adjacent levels coincide ( $\Sigma_k = \Sigma_{k+1}$ ), which forces every higher level down to that one - the hierarchy becomes finite.  $P = NP$  would collapse PH all the way to  $P$ . Because collapse is considered extremely unlikely, "unless PH collapses" is used as strong evidence a statement is false.  $PH \subseteq \text{PSPACE}$ .

### D5. Why is $IP = \text{PSPACE}$ surprising, and what does it tell us about the power of interaction and randomness?

- NP is "one static certificate to a deterministic checker." One might expect adding interaction to help only modestly. But  $IP = \text{PSPACE}$  (Shamir) shows that a *polynomial-time, randomized* verifier chatting over polynomially many rounds with an all-powerful (but untrusted) prover can verify membership in **any** PSPACE problem - vastly beyond NP. The key ingredients are **randomness** (so the prover cannot predict and pre-cheat the verifier's challenges) and **algebraic techniques** (arithmetization of Boolean formulas). It demonstrates that interaction + randomness together are qualitatively more powerful than static proofs, and it underlies zero-knowledge proofs and modern verifiable-computation systems.

## 8. Comparison Tables

### P vs NP vs NP-complete vs NP-hard

| Property                      | P   | NP                        | NP-complete                          | NP-hard                  |
|-------------------------------|-----|---------------------------|--------------------------------------|--------------------------|
| Solvable in poly time?        | Yes | Unknown ( $\supseteq P$ ) | Unknown (if yes $\rightarrow P=NP$ ) | Unknown                  |
| Verify solution in poly time? | Yes | Yes                       | Yes                                  | Not necessarily          |
| Must be in NP?                | Yes | -                         | Yes                                  | No                       |
| Must be a decision problem?   | Yes | Yes                       | Yes                                  | No (can be optimization) |

| Property | P              | NP         | NP-complete   | NP-hard                   |
|----------|----------------|------------|---------------|---------------------------|
| Example  | Sorting, 2-SAT | Subset Sum | 3-SAT, Clique | Optimization TSP, Halting |

## NP vs Co-NP

| Aspect           | NP                                                                                   | Co-NP                          |
|------------------|--------------------------------------------------------------------------------------|--------------------------------|
| Easy to certify  | YES answers                                                                          | NO answers                     |
| Certificate      | satisfying assignment                                                                | falsifying/impossibility proof |
| Complete problem | SAT                                                                                  | UNSAT / TAUTOLOGY              |
| Relation         | complements of each other $L \in \text{Co-NP} \Leftrightarrow \bar{L} \in \text{NP}$ |                                |
| Overlap          | $P \subseteq \text{NP} \cap \text{Co-NP}$ ; equality $\text{NP}=\text{Co-NP}$ open   |                                |

## P vs PSPACE vs EXPTIME

| Class   | Restriction | Time          | Space | Example complete problem    |
|---------|-------------|---------------|-------|-----------------------------|
| P       | poly time   | poly          | poly  | (P-complete: circuit value) |
| NP      | poly verify | poly (nondet) | poly  | SAT                         |
| PSPACE  | poly space  | up to exp     | poly  | TQBF                        |
| EXPTIME | exp time    | exp           | exp   | generalized chess           |

## Karp (many-one) vs Cook (Turing) reduction

| Aspect     | Karp / Many-one ( $\leq_p$ )                       | Cook / Turing ( $\leq_T$ )                |
|------------|----------------------------------------------------|-------------------------------------------|
| Form       | single transform of instance                       | poly-time algorithm using B as an oracle  |
| Calls to B | one                                                | possibly many                             |
| Preserves  | yes $\leftrightarrow$ yes, no $\leftrightarrow$ no | overall answer                            |
| Used for   | NP-completeness proofs                             | broader hardness / relative computability |
| Strength   | more restrictive                                   | more general                              |

## Weakly vs Strongly NP-complete

| Aspect                        | Weakly NP-complete | Strongly NP-complete |
|-------------------------------|--------------------|----------------------|
| Pseudo-poly algorithm exists? | Yes                | No (unless $P=NP$ )  |
| Hard when numbers are small?  | No (becomes easy)  | Yes                  |
| Depends on numeric magnitude? | Yes                | No                   |

| Aspect   | Weakly NP-complete              | Strongly NP-complete                    |
|----------|---------------------------------|-----------------------------------------|
| Examples | Knapsack, Subset Sum, Partition | TSP, 3-SAT, Graph Coloring, Bin Packing |

Monte Carlo vs Las Vegas (randomized algorithms)

| Aspect          | Monte Carlo (BPP/RP)         | Las Vegas (ZPP)             |
|-----------------|------------------------------|-----------------------------|
| Correctness     | may be wrong (bounded prob)  | always correct              |
| Running time    | fixed / deterministic        | random, poly in expectation |
| Error reduction | repeat & majority/AND vote   | n/a (no error)              |
| Example         | Miller-Rabin, Karger min-cut | randomized Quicksort        |

Exact vs Approximation vs Heuristic (responses to NP-hardness)

| Approach       | Guarantee on quality                     | Guarantee on time           | Use when                 |
|----------------|------------------------------------------|-----------------------------|--------------------------|
| Exact (exp)    | optimal                                  | exponential                 | small n                  |
| Approximation  | provable ratio ( $\alpha$ , PTAS, FPTAS) | polynomial                  | need a bound             |
| Heuristic      | none (empirical)                         | usually fast                | large, messy, real-world |
| Pseudo-poly DP | optimal                                  | poly in values              | small numbers            |
| FPT            | optimal                                  | $f(k) \cdot \text{poly}(n)$ | small parameter k        |

## 9. Common Mistakes

1. **"NP means non-polynomial."** It means *nondeterministic polynomial*; NP contains all of P.
2. **Confusing NP-hard and NP-complete.** NP-complete must additionally be *in* NP (verifiable).
3. **Reducing in the wrong direction.** To prove X hard, reduce a *known-hard* problem *into* X, not X into something easy.
4. **Thinking a DP for Knapsack disproves  $P \neq NP$ .** That DP is *pseudo-polynomial* (exponential in bit-length).
5. **Believing P vs NP is solved.** It is open; only widely *believed* to be  $P \neq NP$ .
6. **Assuming all NP-complete problems need the same algorithm.** They are *inter-reducible*, but their natural algorithms differ; equivalence is up to polynomial reductions.
7. **Saying "NP-hard = unsolvable."** Undecidable (Halting)  $\neq$  NP-hard; NP-hard problems can be solved, just (probably) not in polynomial time.
8. **Mixing up time and space classes.** PSPACE restricts *memory*, not time;  $NP \subseteq PSPACE$ .
9. **Thinking Co-NP is disjoint from NP.** They overlap;  $P$  is inside both.
10. **Assuming approximation is always possible.** Some problems are provably hard to approximate (PCP theorem / inapproximability).
11. **Treating 2-SAT and 3-SAT as equally hard.** 2-SAT is in P; 3-SAT is NP-complete.
12. **Confusing "verify" with "solve."** NP is about *verifying* a given certificate quickly, not finding one quickly.

## 10. Edge Cases / Special Cases

- **Undecidable  $\subset$  NP-hard sometimes:** The Halting Problem is NP-hard (everything in NP reduces to it) *and* undecidable, so it is NOT NP-complete (not in NP - no verifier exists).
- **Trivial languages  $\emptyset$  and  $\Sigma^*$**  are technically in P but are excluded from being NP-complete under standard (Karp) reductions because you cannot map yes/no instances into them.
- **2-SAT, 2-coloring, linear programming** feel like NP-complete siblings but are in **P** - restricting structure can drop complexity sharply.
- **Primality:** sat in **NP**  $\cap$  **Co-NP** for decades, then shown to be in **P** (AKS, 2002) - a reminder that "believed hard" can change.
- **Weak vs strong NP-completeness:** Subset Sum with numbers bounded by a polynomial in  $n$  is solvable in polynomial time; with huge numbers it is hard. Same problem, different regimes.
- **Approximation of TSP:** general (non-metric) TSP has *no* constant-factor poly approximation unless  $P=NP$ , but **metric** TSP has a 1.5-approximation (Christofides). The triangle inequality changes everything.
- **FPT vs  $n^k$ :**  $2^k \cdot n$  (FPT) and  $n^k$  (not FPT) look similar for fixed  $k$  but scale completely differently as  $k$  grows; only the first isolates the blow-up to  $k$ .
- **BPP and non-uniformity:** BPP is in the polynomial hierarchy ( $BPP \subseteq \Sigma_2 \cap \Pi_2$ ), even though we do not know if  $BPP \subseteq NP$ .
- **Optimization vs decision:** an optimization problem can be NP-hard while its decision version is NP-complete; interviewers often switch between the two - always restate as a decision problem first.

## 11. How to Explain in Interview

"Complexity theory classifies problems by the resources - mainly time and space - they fundamentally need. **P** is problems we can *solve* in polynomial time. **NP** is problems where we can *verify* a proposed solution in polynomial time - like checking a filled Sudoku is easy even if solving it is hard. **NP-complete** problems are the hardest ones in NP: SAT, 3-SAT, TSP, Knapsack. They are all inter-reducible, so if any one had a fast algorithm, all of NP would - that is the famous open P vs NP question, and the belief is  $P \neq NP$ . **NP-hard** is 'at least as hard as NP' but not necessarily verifiable or even decidable. In practice, when I hit an NP-hard problem like scheduling or routing, I don't chase an exact algorithm - I use an approximation with a provable ratio, or a heuristic, or exploit small numbers with a pseudo-polynomial DP. Recognizing the complexity class tells me which strategy to reach for."

Keep it to ~45 seconds: **define P and NP via solve-vs-verify, mention NP-complete + reductions + P vs NP, then land the practical payoff (approximation/heuristics).**

## 12. Quick Revision Notes

### Key definitions

- **P:** solvable in poly time (deterministic).
- **NP:** yes-answer verifiable in poly time (has a certificate).
- **Co-NP:** no-answer verifiable in poly time.
- **NP-hard:** every NP problem reduces to it ( $\geq$  all of NP).

- **NP-complete:** in NP *and* NP-hard.
- **PSPACE:** poly memory, any time.
- **BPP:** poly time + randomness, bounded two-sided error.
- **FPT:**  $f(k) \cdot \text{poly}(n)$  time.
- **Pseudo-poly:** poly in numeric *value*, not bit-length.

### Important points

- $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$ ;  $P \subseteq \text{Co-NP} \subseteq PSPACE$ .
- SAT = first NP-complete (Cook-Levin, tableau encoding).
- 2-SAT  $\in P$ , 3-SAT NP-complete.
- Reduce *known-hard*  $\rightarrow$  *your problem* to prove hardness.
- One NP-complete in  $P \Rightarrow P = NP$ .
- Savitch:  $NPSpace = PSPACE$ .  $IP = PSPACE$ .  $NP = PCP(\log n, O(1))$ .

### Common comparisons

- NP-hard vs NP-complete (in NP or not).
- NP vs Co-NP (certify yes vs no).
- Weak vs strong NP-completeness (pseudo-poly or not).
- Monte Carlo vs Las Vegas (maybe wrong vs random time).
- Karp vs Cook reductions (one transform vs oracle calls).

### Must-remember facts

- P vs NP is OPEN (Clay \$1M); belief  $P \neq NP$ .
- Halting Problem: NP-hard, undecidable, NOT NP-complete.
- Knapsack: NP-complete but has pseudo-poly DP (weakly NPC).
- Metric TSP: 1.5-approx (Christofides); general TSP: no constant-factor approx unless  $P=NP$ .
- Primality is in P (AKS).

### Interview traps

- "NP = non-polynomial" X.
- "NP-hard = unsolvable" X.
- Reversing reduction direction X.
- "Knapsack DP proves  $P=NP$ " X.
- "Co-NP is outside NP" X (they overlap; P in both).

## 13. Practice Tasks

1. **Classify these problems** as P, NP-complete, or NP-hard (not in NP): Dijkstra shortest path, TSP optimization, 3-Coloring, 2-SAT, Halting Problem, Sorting. (Answers: P, NP-hard, NP-complete, P, NP-hard/undecidable, P)
2. **Write the certificate + verifier** (in words or pseudocode) for Subset Sum, Vertex Cover, and Hamiltonian Cycle. Confirm each verifier is polynomial.
3. **Do a reduction on paper:** reduce 3-SAT to Independent Set (triangle-per-clause construction). Verify a small formula maps correctly.

4. **Implement the Subset Sum DP** in Python/C++ and observe its  $O(n \cdot T)$  behavior; then increase  $T$  to a huge value and watch it blow up - *feel* pseudo-polynomiality.
5. **Code a 2-approximation for Vertex Cover** (pick both endpoints of uncovered edges) and empirically compare its output size to a brute-force optimum on small graphs.
6. **Implement Miller-Rabin** primality test; run multiple rounds and observe error probability shrink - a hands-on Monte Carlo / BPP algorithm.
7. **Trace the class containment chain**  $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$  and write one example complete problem for each.
8. **Prove (informally)** that if  $SAT \in P$  then  $P = NP$ , using the reduction argument.
9. **Explain 2-SAT via implication graph**: build the graph for a small 2-CNF, find SCCs, decide satisfiability. Contrast with why the trick fails for 3-SAT.
10. **Design decision**: given a container-packing feature (bin packing), write a one-paragraph memo stating it is NP-hard and proposing First-Fit-Decreasing with its approximation guarantee - practice the senior-engineer framing.

## 14. Final Cheat Sheet

### CORE DEFINITION

Complexity theory classifies problems by intrinsic resource needs (time, space, randomness) and relates them via reductions.

### THE BIG CLASSES

|             |                                                            |
|-------------|------------------------------------------------------------|
| P           | solve in poly time                                         |
| NP          | verify a YES-certificate in poly time ( $P \subseteq NP$ ) |
| Co-NP       | verify a NO-certificate in poly time                       |
| NP-hard     | $\geq$ every NP problem (via reductions)                   |
| NP-complete | in NP AND NP-hard (hardest inside NP)                      |
| PSPACE      | poly memory, unbounded time                                |
| BPP         | poly time + randomness, bounded error                      |

### CONTAINMENTS

$P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$  ( $P \subsetneq EXPTIME$  is proven)  
 $P \subseteq NP \cap Co-NP$ ,  $PH \subseteq PSPACE$

### KEY THEOREMS

Cook-Levin: SAT is NP-complete (encode NTM computation as a formula)  
 Savitch:  $NPSpace = PSPACE$   
 Shamir:  $IP = PSPACE$   
 PCP:  $NP = PCP(\log n, O(1)) \rightarrow$  hardness of approximation

### WHY IT MATTERS

Tells you when to stop seeking exact fast algorithms and switch to approximation / heuristics / special-case (small numbers, small k).

### MOST ASKED QUESTIONS

- NP-hard vs NP-complete? (NPC also in NP)
- Which way do reductions go? (known-hard  $\rightarrow$  your problem)
- Why 2-SAT in P but 3-SAT NP-complete?
- Does Knapsack DP prove  $P=NP$ ? (No - pseudo-polynomial)

- Is  $P = NP$ ? (Open; believed No)

#### COMMON COMPARISONS

NP vs Co-NP | NP-hard vs NP-complete | Karp vs Cook reduction  
weak vs strong NP-completeness | Monte Carlo vs Las Vegas  
P vs PSPACE vs EXPTIME | approximation vs heuristic vs exact

#### ONE-LINE INTERVIEW ANSWER

"P is solve-fast, NP is verify-fast; NP-complete problems are the hardest in NP and all inter-reducible, so P vs NP asks whether verifying-fast implies solving-fast - believed no, still open."

---

*End of guide - Computational Complexity (TOC), placement interview edition.*